

■ LEVEL 16 • HEAP / PRIORITY QUEUE

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Repeatedly need current minimum/maximum. A heap exposes the extreme without fully sorting everything.
Reference Examples	K Largest • K Smallest • Top K Frequent • Merge K Sorted Lists • Meeting Rooms • Dijkstra
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 16 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Kth Largest Element in an Array	Min Heap	#215
2	Top K Frequent Elements	HashMap + Heap	#347
3	K Closest Points to Origin	Heap	#973
4	Merge K Sorted Lists	Min Heap	#23
5	Find Median from Data Stream	Two Heaps	#295
6	Last Stone Weight	Max Heap	#1046
7	Kth Largest Element in an Array	Min Heap of K	215
8	Top K Frequent Elements	Frequency + Heap	347
9	Last Stone Weight	Max Heap	1046
10	Merge K Sorted Lists	Min Heap	23

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Kth Largest Element in an Array

Pattern: Min Heap

LeetCode: #215

■ Input	integer array and k
■ Output	kth largest value
■ Constraints	$1 \leq k \leq n$
■ Main Idea / Pattern	Keep a min heap of size k. The heap top is the kth largest.
■ Time Complexity	$O(n \log k)$
■ Space Complexity	$O(k)$

■ Optimal C++ Code

```
int findKthLargest(vector<int>& nums, int k) {  
    priority_queue<int, vector<int>, greater<int>> pq;  
  
    for (int x : nums) {  
        pq.push(x);  
        if (pq.size() > k)  
            pq.pop();  
    }  
    return pq.top();  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Top K Frequent Elements

Pattern: HashMap + Heap

LeetCode: #347

■ Input	integer array and k
■ Output	k most frequent values
■ Constraints	answer size k
■ Main Idea / Pattern	Count frequencies, then keep the k highest frequencies in a min heap.
■ Time Complexity	$O(n \log k)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> freq;
    for (int x : nums) freq[x]++;
    using P = pair<int, int>;
    priority_queue<P, vector<P>, greater<P>> pq;
    for (auto& [x, f] : freq) {
        pq.push({f, x});
        if (pq.size() > k)
            pq.pop();
    }
    vector<int> ans;
    while (!pq.empty()) {
        ans.push_back(pq.top().second);
        pq.pop();
    }
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 K Closest Points to Origin

Pattern: Heap

LeetCode: #973

■ Input	points and k
■ Output	k points closest to origin
■ Constraints	$k \leq$ number of points
■ Main Idea / Pattern	Maintain a max heap of size k so the farthest among selected points can be removed.
■ Time Complexity	$O(n \log k)$
■ Space Complexity	$O(k)$

■ Optimal C++ Code

```
vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
    using P = pair<long long, int>;
    priority_queue<P> pq;

    for (int i = 0; i < points.size(); i++) {
        long long d = 1LL * points[i][0] * points[i][0] +
                     1LL * points[i][1] * points[i][1];

        pq.push({d, i});
        if (pq.size() > k)
            pq.pop();
    }

    vector<vector<int>> ans;
    while (!pq.empty()) {
        ans.push_back(points[pq.top().second]);
        pq.pop();
    }
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Merge K Sorted Lists

Pattern: Min Heap

LeetCode: #23

■ Input	k sorted linked lists
■ Output	one sorted linked list
■ Constraints	total nodes N
■ Main Idea / Pattern	Put the head of each list into a min heap and repeatedly extract the smallest node.
■ Time Complexity	$O(N \log k)$
■ Space Complexity	$O(k)$

■ Optimal C++ Code

```
ListNode* mergeKLists(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b) {
        return a->val > b->val;
    };

    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);

    for (ListNode* node : lists)
        if (node) pq.push(node);

    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (!pq.empty()) {
        ListNode* node = pq.top();
        pq.pop();

        tail->next = node;
        tail = node;

        if (node->next)
            pq.push(node->next);
    }

    return dummy.next;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Find Median from Data Stream

Pattern: Two Heaps

LeetCode: #295

■ Input	stream of integers
■ Output	median after each insertion
■ Constraints	standard stream operations
■ Main Idea / Pattern	Max heap stores lower half, min heap stores upper half. Keep sizes balanced.
■ Time Complexity	$O(\log n)$ add
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
class MedianFinder {
    priority_queue<int> low;
    priority_queue<int, vector<int>, greater<int>> high;

public:
    void addNum(int num) {
        low.push(num);

        if (!high.empty() && low.top() > high.top()) {
            high.push(low.top());
            low.pop();
        }

        if (low.size() > high.size() + 1) {
            high.push(low.top());
            low.pop();
        } else if (high.size() > low.size()) {
            low.push(high.top());
            high.pop();
        }
    }

    double findMedian() {
        if (low.size() == high.size())
            return (low.top() + high.top()) / 2.0;
        return low.top();
    }
};
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Last Stone Weight

Pattern: Max Heap

LeetCode: #1046

■ Input	stone weights
■ Output	weight of remaining stone
■ Constraints	positive weights
■ Main Idea / Pattern	Always smash the two largest stones, then push the difference if nonzero.
■■ Time Complexity	$O(n \log n)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
int lastStoneWeight(vector<int>& stones) {
    priority_queue<int> pq;
    for (int x : stones)
        pq.push(x);
    while (pq.size() > 1) {
        int a = pq.top(); pq.pop();
        int b = pq.top(); pq.pop();
        if (a != b)
            pq.push(a - b);
    }
    return pq.empty() ? 0 : pq.top();
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Kth Largest Element in an Array

Pattern: Min Heap of K

LeetCode: 215

Input	nums, k
Output	k-th largest value
Constraints	$1 \leq k \leq \text{nums.size()} \leq 1e5$
Main Idea / Pattern	Keep only the k largest values in a min heap; the heap top is the k-th largest.
Time Complexity	$O(n \log k)$
Space Complexity	$O(k)$

Optimal C++ Code

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for (int x : nums) {
            pq.push(x);
            if (pq.size() > k) {
                pq.pop();
            }
        }
        return pq.top();
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Top K Frequent Elements

Pattern: Frequency + Heap

LeetCode: 347

Input	nums, k
Output	k most frequent values
Constraints	1 <= nums.size() <= 1e5
Main Idea / Pattern	Count frequencies and keep the k highest-frequency values in a min heap.
Time Complexity	O(n log k)
Space Complexity	O(n)

Optimal C++ Code

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int, int> freq;
        for (int x : nums) freq[x]++;
        priority_queue<pair<int, int>,
                      vector<pair<int, int>>,
                      greater<pair<int, int>>> pq;

        for (auto& [x, f] : freq) {
            pq.push({f, x});
            if (pq.size() > k) pq.pop();
        }

        vector<int> ans;
        while (!pq.empty()) {
            ans.push_back(pq.top().second);
            pq.pop();
        }

        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Last Stone Weight

Pattern: Max Heap

LeetCode: 1046

Input	stones
Output	weight of the final remaining stone
Constraints	1 <= stones.size() <= 30
Main Idea / Pattern	Always remove the two largest stones, smash them, and push any positive remainder back.
Time Complexity	O(n log n)
Space Complexity	O(n)

Optimal C++ Code

```
class Solution {
public:
    int lastStoneWeight(vector<int>& stones) {
        priority_queue<int> pq;
        for (int x : stones) pq.push(x);
        while (pq.size() > 1) {
            int a = pq.top(); pq.pop();
            int b = pq.top(); pq.pop();
            if (a != b) pq.push(a - b);
        }
        return pq.empty() ? 0 : pq.top();
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Merge K Sorted Lists

Pattern: Min Heap

LeetCode: 23

Input	lists
Output	one sorted linked list
Constraints	0 <= lists.size() <= 1e4
Main Idea / Pattern	Put each list head into a min heap and repeatedly extract the smallest node.
Time Complexity	O(N log k)
Space Complexity	O(k)

Optimal C++ Code

```
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        auto cmp = [](ListNode* a, ListNode* b) {
            return a->val > b->val;
        };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
        for (ListNode* node : lists) {
            if (node) pq.push(node);
        }
        ListNode dummy(0);
        ListNode* cur = &dummy;
        while (!pq.empty()) {
            ListNode* node = pq.top();
            pq.pop();
            cur->next = node;
            cur = cur->next;
            if (node->next) pq.push(node->next);
        }
        return dummy.next;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.